Cluster
Innovation
Centre

agora. EVENTS

Summer Internship Project Report

Prometheus

Automatic Text Classifier and Content Based Recommender Engine

17.10.2016

Submitted by:
Prashant Sinha
B.Tech. (IT and Mathematical Innovations)
72174

agora. prometheus

agoranhs.com

CONTENTS

ABSTRACT

Providing accurate and relevant recommendations to the users of a service is an important part of the overall user experience and driving sales volume. In this study, we explore various techniques that have been incorporated to build an event aggregation and recommendation platform for health professionals. We address fundamental challenges involved in building such a system, Prometheus, which involved implementation of a Content Based Recommender, Text Classifier and collection of training dataset. Additionally we also expose the shortcomings of the system and present some open problems that should be addressed in future research.

KEYWORDS

Natural Language Processing, Recommender System, Text Classification

ACKNOWLEDGEMENTS

I wish to extend my sincere gratitude to

- Agora Events, where the project was carried out.
- Nicholas Collins, Leif Denby and Jeb Koogler for their critical insights, supervision, alpha testing, and annotating training dataset.
- Devesh Khandelwal, Pragya Jaiswal and Akshay Bajaj for much needed support and guidance.
- The wonderful Open Source Communities on the web for sharing their work and knowledge which has always been vital.

Sincerely,

(Prashant Sinha)

1 INTRODUCTION

Making choices is a natural and essential part of our daily routine. From deciding what to cook for the breakfast, to watching a late-night movie – we are always making choices – knowingly or otherwise. While some of the choices are easy to make: What should I wear today? – others, not so much: Which major should I take at the university? These decisions are easy make if we have a prior knowledge about the available options. However, these turn out trickier wherever we are short of knowledge.

It is indeed possible to learn more about all the choices before making a decision. However, this may become impractical very soon. As an alternative, we regularly seek recommendations from our peers or the advice of experts. This is generally a faster and more practical solution. In addition, this lets us to discover new options or provide alternatives. Reviews about the mobile phones on various technology blogs, recipes for healthier food in the newspapers are all examples of obtaining recommendations.

With the growth of our reliance on various services on internet has grown, so has our expectations from these services. It is natural for us to expect items of relevance while browsing for products on Amazon.in. The Kindle store of Amazon has more than 400,000 titles. Supporting discovery of new titles and recommending titles to the customers is in business interest of Amazon.

However, the historically established source of recommendation: a human agent, is not practical or scalable solution. It is expensive and inefficient to have thousands of curators personally listing books for each individual user on a website.

Given the limit manual recommendation has, there has been a massive effort to develop Automatic Recommender Systems [1] which can generate personalised recommendations for each individual users of a service.

Agora NHS, where the work presented in this paper was carried out, is a platform that aggregates events relevant to the healthcare professionals working at the hospitals under the National Health Service (NHS). The NHS is a public healthcare system for England and the United Kingdom. With hundreds of events relevant to healthcare professionals taking place every day, the lack of a centralised platform for event discovery was realised. Agora has been tasked to develop an event aggregation and recommendation platform which can recommend events that are relevant to a healthcare professional, taking their medical specialty, affiliation and general interests into consideration.

1.1 *Initial Statement of the Problem*

The primary goal of the problem is to find a subset from a given set of events that a given user is most likely to be interested into.

1.2 *Previous Work on Recommender Systems*

Massive amount of research has been done over the last two decades in the fields of automatic recommender systems. The earliest recommender

system, a Librarian, used a hardcoded set of information [2] about various "stereotypes'" book preferences to generate recommendations. It asked a pre-determined set of questions from its user to determine which recommendation to give.

In the 1990s, collaborative filtering systems were developed. Such as system allowed automatic aggregation of recommendation on Usenet. These systems work using behavioural patterns of the users. If a particular user gave high ratings to some five products, and another user was found to be rating in a similar pattern, who'd only rated three out of those five products, the system assumed that this user is more likely to be interested in the rest two, using those as candidates for recommendations. Such a system was found to be surprisingly accurate in services involving a large number of consumers.

Due to the fact that individual users do not obtain any direct knowledge of other users' opinions [3], and the personalised nature of the recommendations, recommender systems using collaborative filtering saw commercial deployments [4] by the end of the 1990s. Amazon.com, for example, adopted a similar recommender technology, utilising purchase history, browsing history and other activities to generate recommendations for other users which they might consider purchasing.

Since such a system can potentially increase the sales volume, the early 2000s saw much more commercial integration [5] and development of more complex systems.

The recommender systems of current generation involve content based approaches from information retrieval [6], bayesian inferences [7], hybrid recommendations [8], etc.

1.3 *Challenges (Part One)*

One shortcoming that the Collaborative Filter based recommender systems have is known as the Cold Start problem [3]. Since the system depends upon the behaviour of the other users, any new service having no previous user data, will not be able to deploy such a system. In addition, the domain specific nature of current problem limits us from using a general purpose recommender system.

Content based recommender systems generate the recommendations taking the metadata of items under consideration [4]. As these metadata are derived from the content itself, recommendations can be generated even when there's no collaborative information is available [3]. Such a system, however, cannot be easily generalised.

The fundamental challenge in developing a Content Based Recommender systems lies in the source of the metadata. Trivially, the metadata is explicitly provided in form of Keywords, Tags and Audience information. Blog writers frequently put these tags themselves hence providing the much needed metadata. Search Engines, such as Google, used to utilise these information for ranking search results [9], which ultimately started the practice of Search Engine Optimisation (SEO) [9] [10].

In the present case, however, metadata extraction is not as simple. The events aggregated from hundreds of sources are unstructured text blobs

lacking any structured metadata. Parsing these text blobs to retrieve event related information is a challenge in itself. For the scope of this paper, however, we assume that the event data is a text document containing the description of the event.

The case discussed in the previous paragraph essentially adds another problem which is formally known as Text Categorisation problem. In text categorisation, the goal is to classify a piece of text, hereby referred as a "document" into a number of predefined categories (tags). These documents may belong within zero or more categories [11].

1.4 *Statement of the Problem*

From the discussion in the previous sections, we define the following problems which are undertaken in this study.

1. We wish to build an automatic content based recommender system which generates recommendations on the basis of document and user metadata.
2. As a prerequisite, we require an automatic text categorisation system which yields metadata used by the recommender engine.

1.5 *Previous Work on Text Categorisation*

Text classification, or categorisation is a problem under Information Retrieval which deals with automatic classification of documents according to their content or attributes. In the present study we are only considering the content classification of the documents.

Machine learning is an exciting and dominant approach taken to solve the Text classification problem where a large amount of pre-classified dataset is used by the algorithm to "learn" the characteristics of the categories. These "trained" algorithms can then be used to classify ("predict") unseen data.

Research for automatic text classification had started as early as 1960s with the study by [1] in which automatic identification of categories was explored while [12] studied the use of hierarchic clustering in Information Retreival. [13] describe a vector space model for automatic indexing.

It wasn't until the 1980s and 90s when enough advancements were made in terms of computation power and other Natural Language Processing concepts that a robust system which can identify and group documents was developed. In [14] information extraction from texts were established, while [15] termed it as text mining. Even with the blurry distinction of domains, Text Classification was formally defined in [6] in 2001.

Various techniques of text classification are being developed, including K-nearest neighbour clustering [16], Latent Semantic Indexing [17] and tf-idf [18]. All these methods require a set of labelled documents for training. We will explore tf-idf classification further in the current study.

1.6 *Challenges (Part Two)*

As with any machine learning problem, a large amount of high quality training dataset is required. Due to the domain specific nature of the undertaken problem, lack of standard dataset is a huge bottleneck [11]. Hence a large part of this problem is attributed to obtaining training dataset with minimal amount of manual labelling required.

With this additional challenge, we describe our methodology in the following section.

2 METHODOLOGY

In this section we will discuss each of the components of the system in detail.

2.1 *Preprocessing*

We begin the task with preprocessing of the dataset. In most of the Natural Language Processing task, it is computationally easier to work on smaller chunks of the text. We call this process of splitting the text *segmentation*. Text segmentation can be on various levels such as section-level segmentation, paragraph-level segmentation, sentence-level segmentation or word-level segmentation. Word-level segmentation is also known as *tokenisation* due to its frequent usage.

After tokenisation of the text, we reduce the token space by selectively pruning the Part of Speech that's been heuristically observed not to contribute to topic categorisation. These include the verbs, adverbs, punctuations etc. This is achieved by performing a Part-of-Speech tagging of the text. Finally, we yield the tokens in form of unigram, bigram and trigram sequences.

2.2 *Text Classification using tf-idf*

2.2.1 *Overview*

Term frequency-inverse document frequency (tf-idf) is a statistical method which reflects the importance of individual words in a document with respect to a particular category [19]. It was introduced in a 1972 as "term specificity" in [20]. This method has been successfully used for text classification, even though its theoretical foundation has not been established yet [21] [22] [18].

tf-idf works because the English language has been observed to follow a Zipfian distribution [23]. Zipfian distribution originates from Zipf's law – an empirical observation which states that in a given corpus of natural language terms, the frequency of any term is inversely proportional to its rank in the frequency table [18]. Examples of such terms in English language are "the", "of", "and", etc. Additionally, these terms, while important syntactically for correct grammar, do not carry topic-related information [20].

TERM FREQUENCY This refers to the statistical measure of frequency of a term in a given document. Formally we denote, the term frequency of a term t in a document d by $\mathrm{tf}(t, d)$.

INVERSE DOCUMENT FREQUENCY This is a statistical measure of how much information the word conveys [18]. In other words, it is used to measure how common or rare is a word. We denote the inverse document frequency of a term t in a set of documents D by $\mathrm{idf}(t, D)$.

TERM FREQUENCY-INVERSE DOCUMENT FREQUENCY It is the product of the above two statistical values. We calculate the tf-idf score of a term t in a document d where $d \in D$ as

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D) \tag{1}$$

2.2.2 *Training: Building the Weight Matrix*

Given a set of text blobs B and a set of labels L, we define the ordered pair

$$\langle b, l \rangle \in B \times L \tag{2}$$

as a document. The set of all available documents, denoted by D where

$$D \subset B \times L \tag{3}$$

is known as a training dataset. For the sake of brevity, we will refer to the blob and the label implicitly as a property of document.

We can further establish that each document $d \in D$ is a multi-set of terms, denoted by t. We can define the vocabulary V as a union set of all the unique terms t in D. That is

$$V = \bigcup_{i=1}^{N} \{t \mid t \in d_i\} \quad \forall\, d_i \in D, N = n(D) \tag{4}$$

In a vector space model we can now treat each of the documents as a vector [24] [13] in v-dimensional space, where v is the cardinality of the vocabulary V.

Hence, we can frame the entire dataset D into a frequency matrix $F = v \times N$. The v component of each document vector is assigned a number which quantifies its importance within that document. If a term is not found in the document, the corresponding component is assigned zero.

We calculate the tf-idf scores using the following function:

$$\text{tfidf}(t, d, D) = f_{t,d} \cdot \log \frac{N}{|d \in D : t \in d|} \tag{5}$$

The weight matrix W is the matrix obtained by element-wise application of 5 on F.

2.2.3 *Prediction: Text classification*

Given the a document d_u, we can find its classification score as a vector dot product of the document vector $\vec{v}_d$ and each weight vector from W [24]. That is:

$$\text{score}(d_u, d_i) = \vec{v}_{d_\text{u}} \cdot \vec{w}_i \tag{6}$$

where i corresponds to the document index in W and $\vec{w}_i \in W$.

2.3 *Content based recommendation using Vector Space Model*

Content based recommenders, as described earlier, utilise metadata of the items as well as the users to generate the personalised recommendations

[4]. As is evident, such a system would require a scheme to map various intangible attributes to a set of quantifiable values [3]. We call this set an attribute vector.

As previously introduced in 2.2.3, in a vector space model, every entities involved in the recommender domain are represented as vectors. Each dimension of the vector corresponds to a separate attribute, hence the term *attribute vector* is used. Given two attribute vectors $\vec{a}_1$ and $\vec{a}_2$, we can calculate the relevance or similarity as their vector dot product. This is formally known as *cosine similarity*.

$$\begin{aligned} \text{similarity}(\vec{a}_1, \vec{a}_2) &= \vec{a}_1 \cdot \vec{a}_2 \\ &= \frac{\sum_{i=1}^{N} a_{1i} \cdot a_{2i}}{\sqrt{\sum_{i=1}^{N} a_{1i}^2} \cdot \sqrt{\sum_{i=1}^{N} a_{2i}^2}} \end{aligned} \tag{7}$$

The fundamental challenge at this stage lies in selection of the attributes that the model is based on [11]. This is an important step because a robust attribute selection directly influences the quality of recommendations. In the present study, this process proved fairly straightforward. We use each medical specialty as a unit-range dimension of the attribute vector. The user profile information of the healthcare professionals consists of their current and past affiliations, their specialties and the specialties they declare they're interested in and several other attributes. We map these information to the attribute vector to obtain a User Attribute Vector. Similarly, we can train the text classifier system to categorise the event descriptions across the attribute vector.

By performing the vectorisation, we're essentially mapping each event and the user in a vector space. This allows us to calculate the relevance score of any pair of user and event using equation 7.

2.4 *Training data collection via seeded crawling of Wikipedia*

As described earlier in 1.6, the result of a machine learning algorithm is directly related to the quality of training dataset [11]. Unfortunately there is no standard dataset available for a tf-idf categorisation of medical specialties. This prompted the need for building our own dataset.

Building a high quality labeled dataset by manual labelling is the most trivial but expensive approach. Given the time-limited nature of the project as well as the constraints, we wished to minimise the amount of manual labour involved in producing a suitable dataset.

2.4.1 *Overview*

A novel approach taken to collect labeled data from public sources such as Wikipedia was developed. The workflow is described as follows:

1. We begin with collecting some *seed* urls on Wikipedia which are directly related to the categories we are interested in. This is the only manual process in the whole scheme.

2. A web crawler is deployed which starts from the above seeds, and follows all the links to other wiki pages recursively until some predefined limit is reached.
3. A directed knowledge-graph is built for the crawled urls. The nodes of this graph are the urls, while the edges are defined if the url contains a hyperlink to the other url. We can optionally assign a normalised weight to this edge as a function of location of the hyperlink in the page: a link found earlier and more frequently is given a higher weight.
4. Using PageRank algorithm the nodes of the knowledge-graph are further assigned ranks.
5. We add our initial category labels as annotations to the seed url nodes in the knowledge-graph.
6. Using *Variational Bayesian* [7] modelling, we propagate the labels across the whole network [25], hence generating probabilistically labeled dataset.

2.4.2 *Network modelling for estimating labels*

We develop this model with an initial assumption that each node in the knowledge-graph, G_k, is built from a set of urls U and discovered links, L, that is:

$$G_k = (U, L) \tag{8}$$

Additionally, given a set of annotations A, each node of the graph G_k is a n-dimensional vector, where n is the cardinality of the set A, that is $n = n(A)$.

Here, we define the nodes, U, as an ordered triplet:

$$U = \{\langle u_i, w_i, \vec{a}_i \rangle\} \tag{9}$$

where u_i is a url, w_i is the weight of nodes calculated using the iterative PageRank algorithm [26], and $\vec{a}_i$ is the annotation vector where each dimension corresponds to the elements in A.

PAGERANK To calculate w_i from a given uninitialised knowledge-network G_k using iterative PageRank algorithm [26], we assume an initial probability distribution of the nodes, that is:

$$\text{Rank}(u_i; 0) = \frac{1}{N} \tag{10}$$

where N is the total number of nodes, $N = n(U)$.

In each time step

$$\text{Rank}(u_i; t+1) = \frac{1-d}{N} + d \cdot \sum_{u_j \in \text{IL}(u_i)} \frac{\text{Rank}(u_j; t)}{\text{OL}(u_j)} \tag{11}$$

where the $\text{IL}(u_j)$ and $\text{OL}(u_j)$ are sets of inbound and outbound hyperlinks from the url u_j, respectively, and d denotes the *damping factor*, a constant value.

Since the equation 11 describes an iterative method, it is not guaranteed to converge. In our case, however, we observed convergence hence making the method suitable.

Having obtained the knowledge-network, G_k, our last problem is to generate the annotations $\vec{a}_i$ for the remaining nodes of the network.

VARIATIONAL MESSAGE PASSING Variational message passing (VMP) is an approximate inference technique for Bayesian networks [7]. Given a set of hidden annotations A_H and observed annotations A_V, the goal of an approximate inference is to assign a lower-bound to the probability that a given knowledge graph is in configuration A_V. Assuming some probability distribution Q, we can define our lower bound as:

$$L(Q) = \sum_H Q(A_H) \ln \frac{P(A_H, A_V)}{Q(A_H)} \tag{12}$$

where $P(A_H, A_V)$ denotes the probability of A_H and AV configurations of G_K.

The likelihood is given by the lower bound and the relative entropy between P and Q. The relative entropy, or Kullback-Leibler divergence is a measure of the difference between two probability distributions [27], and is given by:

$$D_{\mathrm{KL}}(P\|Q) = \sum_i P(i) \ln \frac{P_i}{Q_i} \tag{13}$$

Hence, we have the annotation vector for a node u_i, given by:

$$\vec{a}_i = \{D_{\mathrm{KL}}(P_{\hat{a}_j}\|Q_{\hat{a}_j}) + L(Q_{\hat{a}_j})|\hat{a}_j \in A\} \tag{14}$$

where $\hat{a}_j$ denotes the dimension in the attribute vector space A.

3 IMPLEMENTATION AND DISCUSSIONS

In a software development project, while the algorithms are of essence, several additional constraints has to be factored in due to practicality, cost and performance of the software package. In this section we will discuss several constraints and workarounds which had to be factored in during the development process.

3.1 *Performance*

Performance of this project is of a critical importance. The software is intended to run on a web-server handling more than a thousand requests per second. In light of the volume of traffic, even a sub-millisecond latency could result in tens of thousands of dropped or delayed requests. This is commercially infeasible and provides a bad user experience.

While implementing the algorithm, performance was tested as a function of CPU usage, Memory usage and Latency. Several optimisation techniques utilised were:

1. Fast lookups using a Bloom Filter [28] based existence testing. To calculate the text categorisation vector, it is important to first obtain the values of terms from rank matrix. A Bloom Filter is a data-structure which is used for memory efficient queries of set element existence in constant time. By utilising a bloom filter to check the existence of term in the vocabulary before querying the rank matrix, we were able to reduce lookup-misses.
2. In-memory storage of rank matrix. The primary memory is order of magnitude faster in terms of recall latencies. By minimising the lookup misses in previous step, we are able to prevent the rank matrix from being swapped out by the operating system [29].

3.2 *User Interface*

While the software is a command-line utility, it is still essential to have a friendly user interface which minimises the potential errors by user. The software is developed as a *Django* plugin and provides management commands to preprocess, train, and deploy the model. Django is a popular web development framework written in Python Programming language.

4 CONCLUSION

The focus of this research was developing a sufficiently accurate and feasible solution for automatic content based recommender engine, Prometheus. In addition to that, commercial and practical constraints were essential at all points during the development process. Our findings from this study indicate various fundamental issues faced while developing a non trivial system. We are able to show that most of the steps required to build such a system can be automated. Moreover, we show that we can achieve a sufficient result from the developed scheme. We do realise, however, that the method described in this study are highly domain specific and may not be generalised to other systems.

REFERENCES

[1] R. A. Fairthorne, "Automatic Retrieval of Recorded Information," *The Computer Journal*, vol. 1, pp. 36–41, jan 1958.

[2] E. Rich, "User Modeling via Stereotypes *," *COGNITIVE SCIENCE*, vol. 3, pp. 329–354, 1979.

[3] I. Fernández-Tobías, M. Braunhofer, M. Elahi, F. Ricci, and I. Cantador, "Alleviating the new user problem in collaborative filtering by exploiting personality information," *User Modeling and User-Adapted Interaction*, vol. 26, pp. 221–255, jun 2016.

[4] L. Roy and R. J. Mooney, *Content-based book recommendation using learning for text categorization*. 1999.

[5] "Facebook, Pandora Lead Rise of Recommendation Engines - TIME," *TIME.com*, 2010.

[6] F. Sebastiani, "Machine Learning in Automated Text Categorization," 2001.

[7] J. Baez and T. Fritz, "A Bayesian characterization of relative entropy," *Theory and Application of Categories*, vol. 29, pp. 421–456, 2014.

[8] P. Gupta, A. Goel, J. Lin, A. Sharma, D. Wang, and R. Zadeh, "WTF," in *Proceedings of the 22nd international conference on World Wide Web - WWW '13*, (New York, New York, USA), pp. 505–514, ACM Press, 2013.

[9] B. Pinkerton, *Finding What People Want: Experiences with the WebCrawler*. The Second International WWW Conference Chicago, 2000.

[10] C. Doctorow, *Metacrap: Putting the torch to seven straw-men of the meta-utopia*. e-LearningGuru, 2001.

[11] J. D. Ullman, "Data Mining," 2007.

[12] N. Jardine and C. van Rijsbergen, "The use of hierarchic clustering in information retrieval," *Information Storage and Retrieval*, vol. 7, pp. 217–240, dec 1971.

[13] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, pp. 613–620, nov 1975.

[14] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, pp. 513–523, jan 1988.

[15] G. Salton and M. J. McGill, *Introduction to modern information retrieval*. McGraw-Hill, 1983.

[16] S. T. Dumais, "Latent semantic analysis," *Annual Review of Information Science and Technology*, vol. 38, pp. 188–230, sep 2005.

[17] N. S. Altman, "An Introduction to Kernel and Nearest-Neighbor Nonparametric Regression," *The American Statistician*, vol. 46, pp. 175–185, aug 1992.

[18] H. C. Wu, R. W. P. Luk, K. F. Wong, and K. L. Kwok, "Interpreting TF-IDF term weights as making relevance decisions," *ACM Transactions on Information Systems*, vol. 26, pp. 1–37, jun 2008.

[19] S. Robertson, "Understanding inverse document frequency: on theoretical arguments for IDF," *Journal of Documentation*, vol. 60, pp. 503–520, oct 2004.

[20] K. SPARCK JONES, "A STATISTICAL INTERPRETATION OF TERM SPECIFICITY AND ITS APPLICATION IN RETRIEVAL," *Journal of Documentation*, vol. 28, pp. 11–21, jan 1972.

[21] C. D. Manning, P. Raghavan, and H. Schutze, "Scoring, term weighting, and the vector space model," in *Introduction to Information Retrieval*, pp. 100–123, Cambridge: Cambridge University Press, 2008.

[22] A. A. Goodrum, "Image Information Retrieval: An Overview of Current Research," *Informing Science*, vol. 3, no. 2, 2000.

[23] D. M. W. Powers, "Applications and Explanations of Zipf's Law," 1998.

[24] kak.txo.org, "IR - TFxIDF," 2015.

[25] J. Yedidia, W. Freeman, and Y. Weiss, "Constructing Free-Energy Approximations and Generalized Belief Propagation Algorithms," *IEEE Transactions on Information Theory*, vol. 51, pp. 2282–2312, jul 2005.

[26] S. Brin and L. Page, "The anatomy of a large-scale hypertextual Web search engine," *Computer Networks and ISDN Systems*, vol. 30, pp. 107–117, apr 1998.

[27] S. Kullback and R. A. Leibler, "On Information and Sufficiency," *The Annals of Mathematical Statistics*, vol. 22, pp. 79–86, mar 1951.

[28] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, pp. 422–426, jul 1970.

[29] M. Ahmadi and S. Wong, "A Cache Architecture for Counting Bloom Filters," in *2007 15th IEEE International Conference on Networks*, pp. 218–223, IEEE, nov 2007.

DECLARATION OF CONFLICTING INTEREST

This document describes work undertaken as part of an ongoing software development project at Agora Events, London, UK (the "firm"). The author is an employee at the firm during the time this document was written. The views and opinions expressed therein remain the sole responsibility of the author, and do not necessarily represent those of the firm.